Rubber_band_analysis

Analysis of Field Data and AI-AI in Rubber-band dynamics

PENPAL Project

2025-11-22

Contents

1 1. Data Loading and Preprocessing 2
- 1.1 1.1 Load Data . . . 2
- 1.2 1.2 Data Preprocessing . . . 2
- 1.3 1.3 Compute Lagged Variables and Cosine . . . 4

2 2. Temporal Dynamics Analysis 5
- 2.1 2.1 Stage-Based Analysis (Early/Middle/Late) . . . 5
- 2.2 2.2 Stage Valence Gap Plot . . . 6
- 2.3 2.3 Turn-by-Turn Valence Gap Trajectory . . . 8
- 2.4 2.4 Rubber-Band Dynamics . . . 9
- 2.5 2.5 Turn-by-Turn Rubber-Band Dynamics . . . 10
- 2.6 2.6 Rubber-Band Dynamics Plot . . . 12
- 2.7 2.7 Turn-Level Rubber-Band Dynamics Plot . . . 13

3 3. Permutation tests 15
- 3.1 3.1 Stage-level Permutation Test . . . 15
- 3.2 3.2 Brutal Stage-Level Permutation . . . 16
- 3.3 3.2 Global Shuffle Stage-Level Test . . . 17
- 3.4 3.3 Turn-Level Permutation Test . . . 18

4 4. Cosine similarity alignment 19
- 4.1 4.1 Cosine Similarity Distribution . . . 19
- 4.2 4.2 Cosine Similarity Mixed Model . . . 20
- 4.3 4.3 Cosine Similarity Linear Plot . . . 21
- 4.4 4.4 Cosine Similarity Over Time . . . 22
- 4.5 4.5 Cosine rubber band . . . 23
- 4.6 4.6 Turn level Cosine Permutation test . . . 24
- 4.7 4.7 Turn Level Cosine RB Linear plot . . . 25
- 4.8 4.8 Correlation Between Cosine Distance and Valence Rubber-Band . . . 26

5 5. Supplementary: Interaction Effects 30
- 5.1 5.1 Starter Interaction Effects . . . 30
  - 5.1.1 5.1.1 Valence Gap by Starter . . . 30
  - 5.1.2 5.1.2 Cosine Distance by Starter . . . 31
  - 5.1.3 5.1.3 Alignment (Cosine Similarity) by Starter . . . 31
- 5.2 5.2 LLM Type Interaction Effects . . . 32
  - 5.2.1 5.2.1 Valence Gap by LLM Type . . . 32
  - 5.2.2 5.2.2 Cosine Distance by LLM Type . . . 33
  - 5.2.3 5.2.3 Alignment (Cosine Similarity) by LLM Type . . . 34

1 1. Data Loading and Preprocessing

1.1 1.1 Load Data

```
# Load dyadic sentiment scores
setwd("/Users/halfidaldal/Documents/PENPAL_analysis")
df_field <- read_parquet("data/TEXT/processed/dyadic_sentiment_scores.parquet")
df_simulated <- read_parquet("data/Berlin/processed/dyadic_sentiment_scores_simulated.parquet")

# Display structure
cat("Field data dimensions:", dim(df_field), "\n")
```

```
## Field data dimensions: 720 14
```

```
cat("Simulated data dimensions:", dim(df_simulated), "\n")
```

```
## Simulated data dimensions: 30 12
```

```
# Show column names
cat("\nField data columns:\n")
```

```
##
## Field data columns:
```

```
print(names(df_field))
```

```
##  [1] "timestamp"            "user"                 "ai"
##  [4] "conversation_id"      "respondent_id"        "interaction_count"
##  [7] "llm_type"             "starter"              "user_embedding"
## [10] "ai_embedding"         "turn"                 "pct_turn"
## [13] "user_sentiment_score" "ai_sentiment_score"
```

```
cat("\nSimulated data columns:\n")
```

```
##
## Simulated data columns:
```

```
print(names(df_simulated))
```

```
##  [1] "turn"                 "user"                 "ai"
##  [4] "client_id"            "workshop_id"          "conversation_id"
##  [7] "timestamp"            "user_embedding"       "ai_embedding"
## [10] "pct_turn"             "user_sentiment_score" "ai_sentiment_score"
```

1.2 1.2 Data Preprocessing

```
# Preprocess field data
df_field <- df_field %>%
  rename(
    ai_valence = ai_sentiment_score,
    user_valence = user_sentiment_score
  ) %>%
  mutate(
    # Token counts
    user_token_count = stri_count_words(user),
    ai_token_count = stri_count_words(ai),
    token_ratio = if_else(ai_token_count == 0, NA_real_,
                          user_token_count / ai_token_count),
```

```r
    # Time parsing
    timestamp = ymd_hms(timestamp),

    # Data source label
    data_source = "Field"
  ) %>%
  # Ensure conversation_id is a character/factor
  mutate(conversation_id = as.character(conversation_id))

# Preprocess simulated data
df_simulated <- df_simulated %>%
  rename(
    ai_valence = ai_sentiment_score,
    user_valence = user_sentiment_score
  ) %>%
  mutate(
    # Token counts
    user_token_count = stri_count_words(user),
    ai_token_count = stri_count_words(ai),
    token_ratio = if_else(ai_token_count == 0, NA_real_,
                          user_token_count / ai_token_count),

    # Time parsing (if available)
    timestamp = if_else(!is.na(timestamp), ymd_hms(timestamp), as.POSIXct(NA)),

    # Data source label
    data_source = "Simulated"
  ) %>%
  mutate(conversation_id = as.character(conversation_id))

# Summary statistics
cat("\n=== Field Data Summary ===\n")
```

```
##
## === Field Data Summary ===
```

```r
df_field %>%
  summarise(
    n_conversations = n_distinct(conversation_id),
    n_turns = n(),
    mean_turns_per_story = n() / n_distinct(conversation_id),
    mean_user_valence = mean(user_valence, na.rm = TRUE),
    mean_ai_valence = mean(ai_valence, na.rm = TRUE)
  ) %>%
  print()
```

```
## # A tibble: 1 x 5
##   n_conversations n_turns mean_turns_per_story mean_user_valence mean_ai_valence
##             <int>   <int>                <dbl>             <dbl>           <dbl>
## 1              72     720                   10            0.0422           0.169
```

```r
cat("\n=== Simulated Data Summary ===\n")
```

```
##
## === Simulated Data Summary ===
```

```r
df_simulated %>%
  summarise(
    n_conversations = n_distinct(conversation_id),
    n_turns = n(),
    mean_turns_per_story = n() / n_distinct(conversation_id),
    mean_user_valence = mean(user_valence, na.rm = TRUE),
    mean_ai_valence = mean(ai_valence, na.rm = TRUE)
  ) %>%
  print()
```

```
## # A tibble: 1 x 5
##   n_conversations n_turns mean_turns_per_story mean_user_valence mean_ai_valence
##             <int>   <int>                <dbl>             <dbl>           <dbl>
## 1               6      30                    5            -0.407          -0.296
```

1.3 1.3 Compute Lagged Variables and Cosine

```r
# Add lagged AI valence for field data
# Note: lag() within group_by ensures no cross-story contamination
# Turn 1 of each story will have NA for lagged values (correct behavior)

df_field <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_valence_lag1 = lag(ai_valence, n = 1),
    user_valence_lag1 = lag(user_valence, n = 1),
  ) %>%
  ungroup()


cosine_sim <- function(x, y) {
  if (is.null(x) || is.null(y) || any(is.na(x)) || any(is.na(y))) {
    return(NA_real_)
  }
  return(sum(x * y) / (sqrt(sum(x^2)) * sqrt(sum(y^2))))
}

df_field <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_embedding_prev = lag(ai_embedding)
  ) %>%
  rowwise() %>%
  mutate(
    cosine_sim_same = cosine_sim(user_embedding, ai_embedding),
    cosine_sim_prev = cosine_sim(user_embedding, ai_embedding_prev)
  ) %>%
  ungroup()

# Add lagged AI valence for simulated data
df_simulated <- df_simulated %>%
```

```r
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_valence_lag1 = lag(ai_valence, n = 1),
    user_valence_lag1 = lag(user_valence, n = 1)
  ) %>%
  ungroup()

# Verify no cross-story contamination
cat("=== Lag Variable Verification ===\n")
```

```
## === Lag Variable Verification ===
```

```r
cat("Field data - Turn 1 with non-NA lag (should be 0):",
    sum(df_field$turn == 1 & !is.na(df_field$ai_valence_lag1)), "\n")
```

```
## Field data - Turn 1 with non-NA lag (should be 0): 0
```

```r
cat("Simulated data - Turn 1 with non-NA lag (should be 0):",
    sum(df_simulated$turn == 1 & !is.na(df_simulated$ai_valence_lag1)), "\n")
```

```
## Simulated data - Turn 1 with non-NA lag (should be 0): 0
```

2 2. Temporal Dynamics Analysis

2.1 2.1 Stage-Based Analysis (Early/Middle/Late)

```r
# Function to compute stage-based valence gap
compute_stage_gaps <- function(df, data_source_label) {
  df %>%
    group_by(conversation_id) %>%
    arrange(turn, .by_group = TRUE) %>%
    mutate(
      turn_idx = row_number(),
      n_turns = n()
    ) %>%
    filter(n_turns >= 3) %>%  # Need at least 3 turns
    mutate(
      stage = ntile(turn_idx, 3),
      valence_gap = abs(user_valence - ai_valence)
    ) %>%
    ungroup() %>%
    group_by(conversation_id, stage) %>%
    summarise(
      mean_gap = mean(valence_gap, na.rm = TRUE),
      n_turns = first(n_turns),
      .groups = "drop"
    ) %>%
    mutate(
      data_source = data_source_label,
      stage_label = case_when(
        stage == 1 ~ "Early",
        stage == 2 ~ "Middle",
        stage == 3 ~ "Late"
```

```r
      ),
      stage_label = factor(stage_label, levels = c("Early", "Middle", "Late")),
      story_length = if_else(n_turns >= median(n_turns),
                             "High  median", "Low < median")
    )
}

# Compute for both datasets
stage_gaps_field <- compute_stage_gaps(df_field, "Field")
stage_gaps_sim <- compute_stage_gaps(df_simulated, "Simulated")

# Combined
stage_gaps_all <- bind_rows(stage_gaps_field, stage_gaps_sim)

# Summary by stage
cat("=== Mean Valence Gap by Stage ===\n")
```

```
## === Mean Valence Gap by Stage ===
```

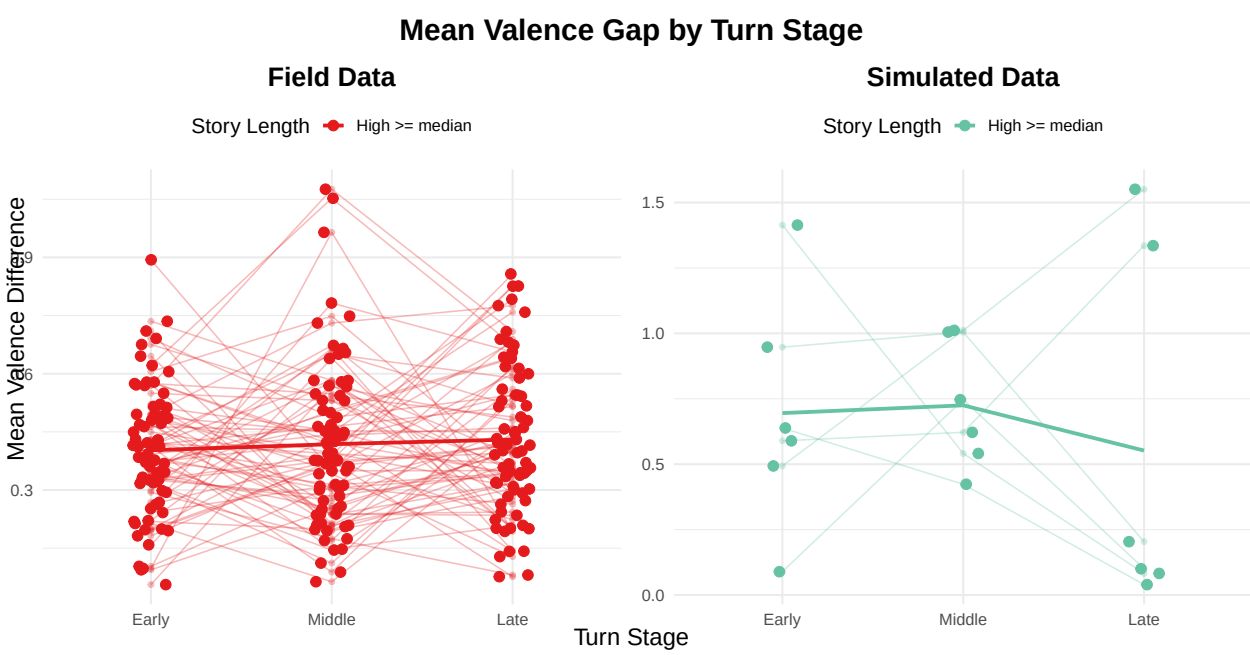
```r
stage_gaps_all %>%
  group_by(data_source, stage_label) %>%
  summarise(
    mean_gap = mean(mean_gap, na.rm = TRUE),
    sd_gap = sd(mean_gap, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 6 x 4
##   data_source stage_label mean_gap sd_gap
##   <chr>       <fct>          <dbl>  <dbl>
## 1 Field       Early          0.403     NA
## 2 Field       Middle         0.418     NA
## 3 Field       Late           0.430     NA
## 4 Simulated   Early          0.695     NA
## 5 Simulated   Middle         0.725     NA
## 6 Simulated   Late           0.552     NA
```

2.2 2.2 Stage Valence Gap Plot

```r
# Field plot
p_stage_field <- ggplot(stage_gaps_field,
                        aes(x = stage_label, y = mean_gap,
                            group = conversation_id, color = story_length)) +
  geom_line(alpha = 0.3) +
  geom_point(alpha = 0.3) +
  stat_summary(fun = mean, geom = "line", aes(group = story_length),
               linewidth = 1.2, position = position_dodge(width = 0.2)) +
  stat_summary(fun = mean, geom = "point",
               size = 3, position = position_dodge(width = 0.2)) +
  scale_color_brewer(palette = "Set1", name = "Story Length") +
  labs(title = "Field Data", x = NULL, y = NULL) +
  theme(
    legend.position = "top",
```

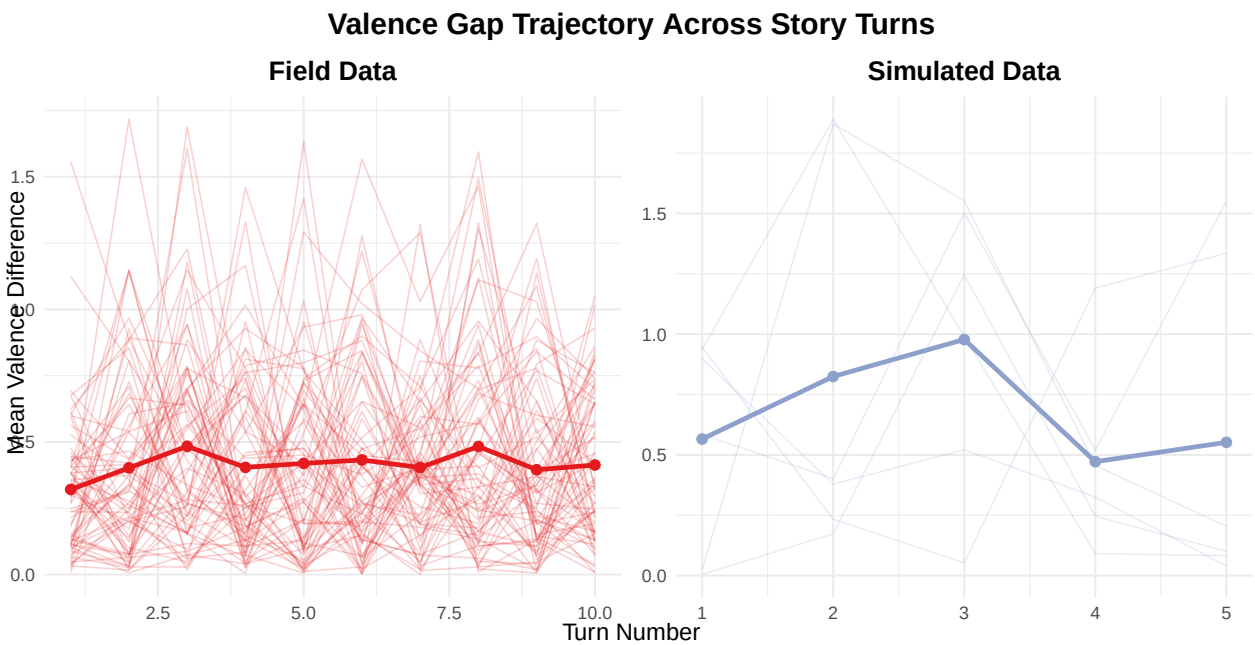
```r
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold")
  )

# Simulated plot
p_stage_sim <- ggplot(stage_gaps_sim,
                      aes(x = stage_label, y = mean_gap,
                          group = conversation_id, color = story_length)) +
  geom_line(alpha = 0.3) +
  geom_point(alpha = 0.3) +
  stat_summary(fun = mean, geom = "line", aes(group = story_length),
               linewidth = 1.2, position = position_dodge(width = 0.2)) +
  stat_summary(fun = mean, geom = "point",
               size = 3, position = position_dodge(width = 0.2)) +
  scale_color_brewer(palette = "Set2", name = "Story Length") +
  labs(title = "Simulated Data", x = NULL, y = NULL) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold")
  )

# Combine
combined_stage <- (p_stage_field | p_stage_sim) +
  plot_annotation(
    title = "Mean Valence Gap by Turn Stage",
    theme = theme(plot.title = element_text(hjust = 0.5, size = 20, face = "bold"))
  )

final_stage <- ggdraw() +
  draw_plot(combined_stage, 0, 0, 1, 1) +
  draw_label("Mean Valence Difference", x = 0.02, y = 0.5, angle = 90, size = 16) +
  draw_label("Turn Stage", x = 0.5, y = 0.02, size = 16)

print(final_stage)
```



```r
ggsave(file.path(GRAPHS_DIR, "stage_valence_gap_comparison.png"),
       final_stage, width = 12, height = 6, dpi = 300)
```

2.3 2.3 Turn-by-Turn Valence Gap Trajectory

```r
# Compute valence gap for each turn
df_field_gap <- df_field %>%
  mutate(valence_gap = abs(user_valence - ai_valence))

df_sim_gap <- df_simulated %>%
  mutate(valence_gap = abs(user_valence - ai_valence))

# Field data plot
p_gap_field <- ggplot(df_field_gap, aes(x = turn, y = valence_gap, group = conversation_id)) +
  geom_line(alpha = 0.2, color = "#E41A1C") +  # Individual stories (faint)
  stat_summary(aes(group = 1), fun = mean, geom = "line",
               color = "#E41A1C", linewidth = 1.5) +  # Aggregate mean (bold)
  stat_summary(aes(group = 1), fun = mean, geom = "point",
               color = "#E41A1C", size = 3) +
  labs(title = "Field Data", x = NULL, y = NULL) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.text = element_text(size = 12)
  )

# Simulated data plot
p_gap_sim <- ggplot(df_sim_gap, aes(x = turn, y = valence_gap, group = conversation_id)) +
  geom_line(alpha = 0.2, color = "#8DA0CB") +  # Individual stories (faint)
  stat_summary(aes(group = 1), fun = mean, geom = "line",
               color = "#8DA0CB", linewidth = 1.5) +  # Aggregate mean (bold)
  stat_summary(aes(group = 1), fun = mean, geom = "point",
               color = "#8DA0CB", size = 3) +
  labs(title = "Simulated Data", x = NULL, y = NULL) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.text = element_text(size = 12)
  )

# Combine
combined_gap_trajectory <- (p_gap_field | p_gap_sim) +
  plot_annotation(
    title = "Valence Gap Trajectory Across Story Turns",
    theme = theme(plot.title = element_text(hjust = 0.5, size = 20, face = "bold"))
  )

final_gap_trajectory <- ggdraw() +
  draw_plot(combined_gap_trajectory, 0, 0, 1, 1) +
  draw_label("Mean Valence Difference", x = 0.02, y = 0.5, angle = 90, size = 16) +
  draw_label("Turn Number", x = 0.5, y = 0.02, size = 16)

print(final_gap_trajectory)
```




```
ggsave(file.path(GRAPHS_DIR, "gap_trajectory_comparison.png"),
       final_gap_trajectory, width = 12, height = 6, dpi = 300)
```

2.4 2.4 Rubber-Band Dynamics

```
# Function to compute rubber-band dynamics
compute_rubberband <- function(df, data_source_label) {
  df %>%
    group_by(conversation_id) %>%
    arrange(turn, .by_group = TRUE) %>%
    mutate(
      turn_idx = row_number(),
      n_turns = n(),
      valence_gap = abs(user_valence - ai_valence)
    ) %>%
    filter(n_turns >= 3) %>%
    mutate(stage = ntile(turn_idx, 3)) %>%
    ungroup() %>%
    group_by(conversation_id, stage) %>%
    summarise(
      mean_gap = mean(valence_gap, na.rm = TRUE),
      n_turns = first(n_turns),
      .groups = "drop"
    ) %>%
    pivot_wider(
      id_cols = c(conversation_id, n_turns),
      names_from = stage,
      values_from = mean_gap,
      names_prefix = "g"
    ) %>%
    filter(!is.na(g1) & !is.na(g2) & !is.na(g3)) %>%
    mutate(
      delta_12 = g2 - g1,  # Change from Early to Middle
```

```r
      delta_23 = g3 - g2,  # Change from Middle to Late
      data_source = data_source_label,
      story_length = if_else(n_turns >= median(n_turns),
                             "High  median", "Low < median")
    )
}

# Compute for both datasets
rb_field <- compute_rubberband(df_field, "Field")
rb_sim <- compute_rubberband(df_simulated, "Simulated")

# Fit linear models
lm_field <- lm(delta_23 ~ delta_12, data = rb_field)
#lm_sim <- lm(delta_23 ~ delta_12 * story_length, data = rb_sim)

cat("=== Rubber-Band Model: Field Data ===\n")
```

```
## === Rubber-Band Model: Field Data ===
```

```r
print(summary(lm_field))
```

```
##
## Call:
## lm(formula = delta_23 ~ delta_12, data = rb_field)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.50008 -0.16134  0.00665  0.14352  0.48625
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.02190    0.02472   0.886    0.379
## delta_12    -0.62797    0.09575  -6.558  7.9e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2093 on 70 degrees of freedom
## Multiple R-squared:  0.3806, Adjusted R-squared:  0.3717
## F-statistic: 43.01 on 1 and 70 DF,  p-value: 7.901e-09
```

```r
cat("\n=== Rubber-Band Model: Simulated Data ===\n")
```

```
##
## === Rubber-Band Model: Simulated Data ===
```

```r
#print(summary(lm_sim))
```

2.5 2.5 Turn-by-Turn Rubber-Band Dynamics

```r
# Function to compute turn-by-turn rubber-band dynamics
compute_rubberband_turns <- function(df, data_source_label) {
  df %>%
    group_by(conversation_id) %>%
    arrange(turn, .by_group = TRUE) %>%
    mutate(
```

```r
      valence_gap = abs(user_valence - ai_valence),
      gap_change = valence_gap - lag(valence_gap),  # Change from previous turn
      gap_change_next = lead(valence_gap) - valence_gap  # Change to next turn
    ) %>%
    filter(!is.na(gap_change) & !is.na(gap_change_next)) %>%  # Need both values
    ungroup() %>%
    mutate(
      data_source = data_source_label,
      n_turns = n_distinct(conversation_id)
    )
}

# Compute for both datasets
rb_turns_field <- compute_rubberband_turns(df_field, "Field")
rb_turns_sim <- compute_rubberband_turns(df_simulated, "Simulated")

# Fit linear models for turn-level rubber-band
lm_turns_field <- lm(gap_change_next ~ gap_change, data = rb_turns_field)
lm_turns_sim <- lm(gap_change_next ~ gap_change, data = rb_turns_sim)

cat("=== Turn-Level Rubber-Band Model: Field Data ===\n")
```

```
## === Turn-Level Rubber-Band Model: Field Data ===
```

```r
print(summary(lm_turns_field))
```

```
##
## Call:
## lm(formula = gap_change_next ~ gap_change, data = rb_turns_field)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.11442 -0.28489 -0.03194  0.26570  1.57926
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.006275   0.017258   0.364    0.716
## gap_change  -0.529129   0.035919 -14.731   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4141 on 574 degrees of freedom
## Multiple R-squared:  0.2743, Adjusted R-squared:  0.2731
## F-statistic:   217 on 1 and 574 DF,  p-value: < 2.2e-16
```

```r
cat("\n=== Turn-Level Rubber-Band Model: Simulated Data ===\n")
```

```
##
## === Turn-Level Rubber-Band Model: Simulated Data ===
```

```r
print(summary(lm_turns_sim))
```

```
##
## Call:
## lm(formula = gap_change_next ~ gap_change, data = rb_turns_sim)
##
```

```
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.0692 -0.5424 -0.1708  0.4181  1.2092
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.09783    0.17752  -0.551    0.589
## gap_change  -0.22171    0.20273  -1.094    0.290
##
## Residual standard error: 0.7527 on 16 degrees of freedom
## Multiple R-squared:  0.06955,    Adjusted R-squared:  0.0114
## F-statistic: 1.196 on 1 and 16 DF,  p-value: 0.2903
```

```r
# Summary statistics
cat("\n=== Turn-Level Rubber-Band Correlation ===\n")
```

```
##
## === Turn-Level Rubber-Band Correlation ===
```

```r
cat("Field correlation (gap_change vs gap_change_next):",
    cor(rb_turns_field$gap_change, rb_turns_field$gap_change_next, use = "complete.obs"), "\n")
```

```
## Field correlation (gap_change vs gap_change_next): -0.5237752
```

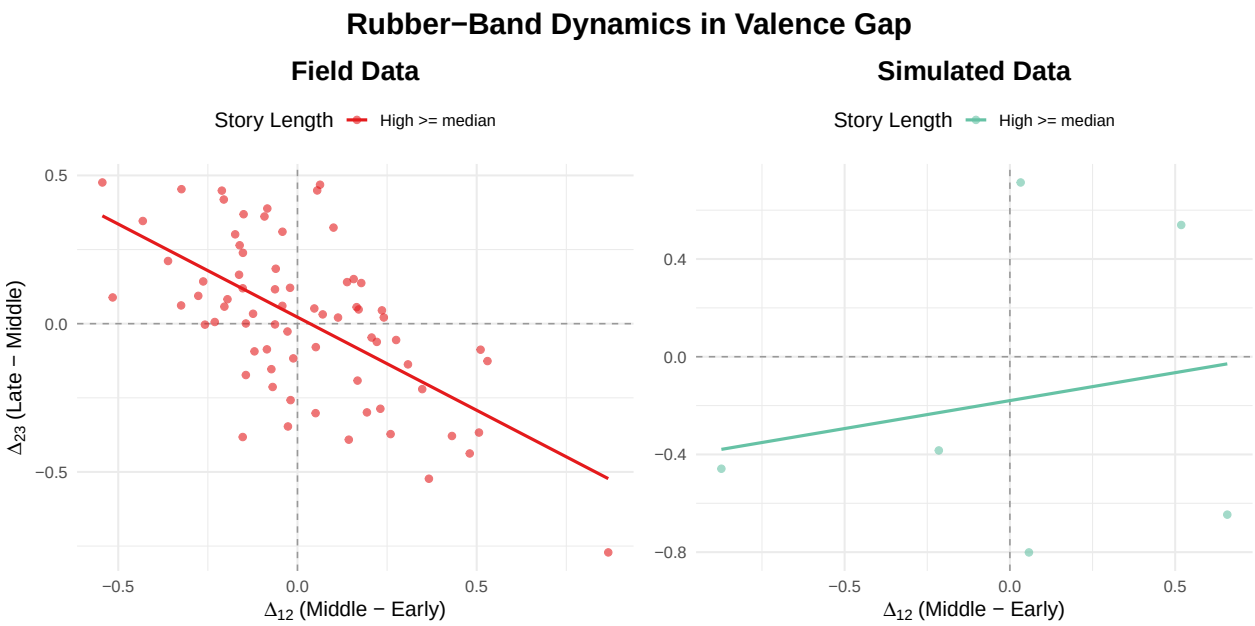
```r
cat("Simulated correlation (gap_change vs gap_change_next):",
    cor(rb_turns_sim$gap_change, rb_turns_sim$gap_change_next, use = "complete.obs"), "\n")
```

```
## Simulated correlation (gap_change vs gap_change_next): -0.2637313
```

2.6 2.6 Rubber-Band Dynamics Plot

```r
# Field plot
p_rb_field <- ggplot(rb_field, aes(x = delta_12, y = delta_23, color = story_length)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.6, size = 2) +
  geom_smooth(method = "lm", aes(linetype = story_length), se = FALSE, linewidth = 1) +
  scale_color_brewer(palette = "Set1", name = "Story Length") +
  scale_linetype_manual(values = c("solid", "dashed"), name = "Story Length") +
  labs(
    title = "Field Data",
    x = expression(Delta[12]~"(Middle - Early)"),
    y = expression(Delta[23]~"(Late - Middle)")
  ) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title = element_text(size = 14, face = "bold")
  )

# Simulated plot
p_rb_sim <- ggplot(rb_sim, aes(x = delta_12, y = delta_23, color = story_length)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.6, size = 2) +
  geom_smooth(method = "lm", aes(linetype = story_length), se = FALSE, linewidth = 1) +
```

```r
  scale_color_brewer(palette = "Set2", name = "Story Length") +
  scale_linetype_manual(values = c("solid", "dashed"), name = "Story Length") +
  labs(
    title = "Simulated Data",
    x = expression(Delta[12]~"(Middle - Early)"),
    y = NULL
  ) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title.x = element_text(size = 14, face = "bold")
  )

# Combine
combined_rb <- (p_rb_field | p_rb_sim) +
  plot_annotation(
    title = "Rubber-Band Dynamics in Valence Gap",
    theme = theme(plot.title = element_text(hjust = 0.5, size = 20, face = "bold"))
  ) &
  theme(legend.position = "top")

print(combined_rb)
```

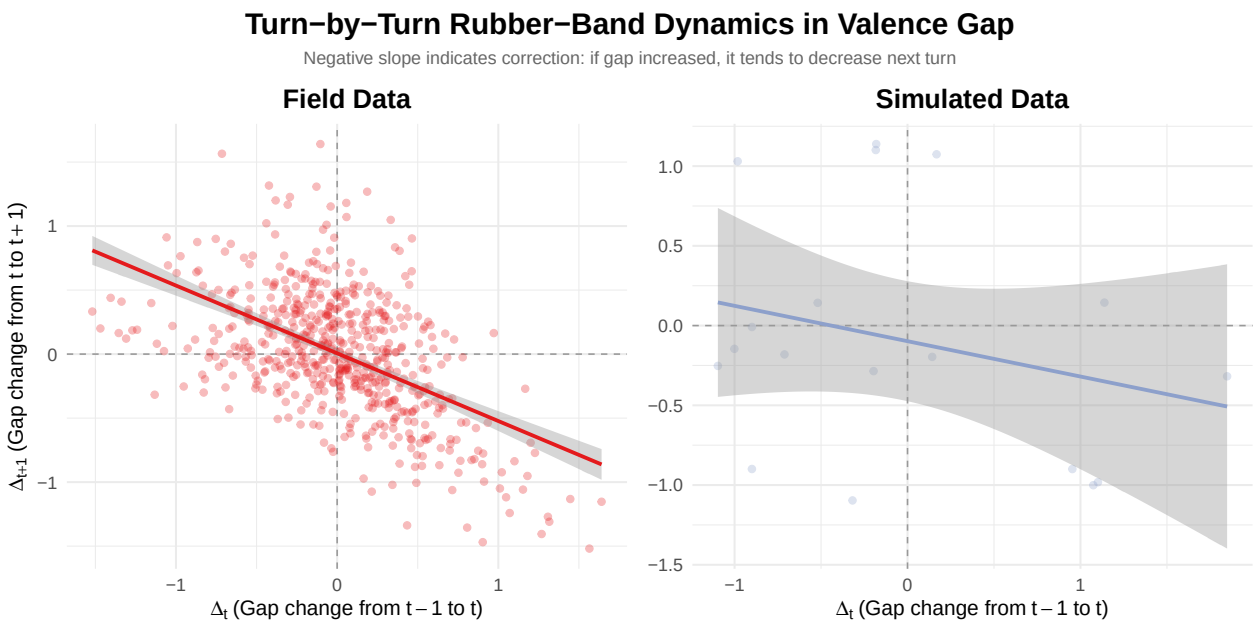



```r
ggsave(file.path(GRAPHS_DIR, "rubberband_dynamics_comparison.png"),
       combined_rb, width = 12, height = 6, dpi = 300)
```

2.7 2.7 Turn-Level Rubber-Band Dynamics Plot

```r
# Field plot
p_rb_turns_field <- ggplot(rb_turns_field, aes(x = gap_change, y = gap_change_next)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.3, size = 2, color = "#E41A1C") +
```

```r
  geom_smooth(method = "lm", se = TRUE, color = "#E41A1C", linewidth = 1.2) +
  labs(
    title = "Field Data",
    x = expression(Delta[t]~"(Gap change from"~t-1~"to"~t*")"),
    y = expression(Delta[t+1]~"(Gap change from"~t~"to"~t+1*")")
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

# Simulated plot
p_rb_turns_sim <- ggplot(rb_turns_sim, aes(x = gap_change, y = gap_change_next)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.3, size = 2, color = "#8DA0CB") +
  geom_smooth(method = "lm", se = TRUE, color = "#8DA0CB", linewidth = 1.2) +
  labs(
    title = "Simulated Data",
    x = expression(Delta[t]~"(Gap change from"~t-1~"to"~t*")"),
    y = NULL
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title.x = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

# Combine
combined_rb_turns <- (p_rb_turns_field | p_rb_turns_sim) +
  plot_annotation(
    title = "Turn-by-Turn Rubber-Band Dynamics in Valence Gap",
    subtitle = "Negative slope indicates correction: if gap increased, it tends to decrease next turn",
    theme = theme(
      plot.title = element_text(hjust = 0.5, size = 20, face = "bold"),
      plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40")
    )
  )

print(combined_rb_turns)
```




```
ggsave(file.path(GRAPHS_DIR, "rubberband_turn_level_comparison.png"),
       combined_rb_turns, width = 12, height = 6, dpi = 300)
```

3 3. Permutation tests

3.1 3.1 Stage-level Permutation Test

```
obs_perm_slope <- coef(lm_field)[2]


set.seed(2025)
n_sims <- 1000
perm_slopes <- replicate(n_sims, {
  df_perm <- stage_gaps_field %>%
    group_by(conversation_id) %>%
    mutate(stage_p = sample(stage)) %>%    # shuffle within each id
    ungroup() %>%
    filter(stage_p %in% 1:3) %>%
    pivot_wider(
      id_cols     = conversation_id,
      names_from  = stage_p,
      values_from = mean_gap,
      names_prefix= "g",
      values_fn   = mean,      # collapse duplicates if any
      values_fill = NA
    ) %>%
    filter(!is.na(g1) & !is.na(g2) & !is.na(g3)) %>%
    transmute(
      d12 = g2 - g1,
      d23 = g3 - g2
    )
    coef(lm(d23 ~ d12, data = df_perm))[2]
})
```

```r
p_stage <- mean(abs(perm_slopes) >= abs(obs_perm_slope))

cat("=== Stage-Level Permutation Test Results ===\n")
```

```
## === Stage-Level Permutation Test Results ===
```

```r
cat("Observed slope:", round(obs_perm_slope, 4), "\n")
```

```
## Observed slope: -0.628
```

```r
cat("Null mean:", round(mean(perm_slopes), 4), "\n")
```

```
## Null mean: -0.4962
```

```r
cat("Null SD:", round(sd(perm_slopes), 4), "\n")
```

```
## Null SD: 0.1064
```

```r
cat("Effect size (z):", round((obs_perm_slope - mean(perm_slopes)) / sd(perm_slopes), 2), "\n")
```

```
## Effect size (z): -1.24
```

```r
cat("P-value:", format.pval(p_stage, digits = 3), "\n\n")
```

```
## P-value: 0.1
```

3.2 3.2 Brutal Stage-Level Permutation

```r
set.seed(2025)
n_sims <- 1000

perm_slopes_stage_brutal <- replicate(n_sims, {
  df_perm_brutal <- df_field_gap %>%
    group_by(conversation_id) %>%
    arrange(turn, .by_group = TRUE) %>%
    mutate(
      # shuffle the *sequence* of gaps inside each story
      valence_gap_perm = sample(valence_gap)
    ) %>%
    ungroup() %>%
    group_by(conversation_id) %>%
    mutate(
      turn_idx = row_number(),
      n_turns = n(),
      stage = ntile(turn_idx, 3)
    ) %>%
    group_by(conversation_id, stage) %>%
    summarise(
      mean_gap = mean(valence_gap_perm, na.rm = TRUE),
      n_turns = first(n_turns),
      .groups = "drop"
    ) %>%
    pivot_wider(
      id_cols = c(conversation_id, n_turns),
      names_from = stage,
      values_from = mean_gap,
```

```r
      names_prefix = "g"
    ) %>%
    filter(!is.na(g1), !is.na(g2), !is.na(g3)) %>%
    mutate(
      delta_12 = g2 - g1,
      delta_23 = g3 - g2
    )

  coef(lm(delta_23 ~ delta_12, data = df_perm_brutal))[2]  # slope only
})

p_stage_brutal <- mean(abs(perm_slopes_stage_brutal) >= abs(obs_perm_slope))

cat("=== Stage-Level Permutation Test Results - Shuffle Turn-Level Valence Gap first ===\n")
```

```
## === Stage-Level Permutation Test Results - Shuffle Turn-Level Valence Gap first ===
```

```r
cat("Observed slope:", round(obs_perm_slope, 4), "\n")
```

```
## Observed slope: -0.628
```

```r
cat("Null mean:", round(mean(perm_slopes_stage_brutal), 4), "\n")
```

```
## Null mean: -0.57
```

```r
cat("Null SD:", round(sd(perm_slopes_stage_brutal), 4), "\n")
```

```
## Null SD: 0.112
```

```r
cat("Effect size (z):", round((obs_perm_slope - mean(perm_slopes_stage_brutal)) / sd(perm_slopes_stage_b
```

```
## Effect size (z): -0.52
```

```r
cat("P-value:", format.pval(p_stage_brutal, digits = 3), "\n\n")
```

```
## P-value: 0.302
```

3.3 3.2 Global Shuffle Stage-Level Test

```r
set.seed(2025)
df_global <- stage_gaps_field%>%
  mutate(stage_g = sample(stage)) %>%      # shuffle across all IDs
  group_by(conversation_id, stage_g) %>%
  summarise(mean_gap = mean(mean_gap), .groups = "drop") %>%  # collapse any duplicates
  pivot_wider(
    id_cols     = conversation_id,
    names_from  = stage_g,
    values_from = mean_gap,
    names_prefix= "g",
    values_fn   = mean,
    values_fill = NA
  ) %>%
  filter(!is.na(g1) & !is.na(g2) & !is.na(g3)) %>%
  transmute(
    d12 = g2 - g1,
    d23 = g3 - g2
  )
```

```r
lm_global_shuffle <- lm(d23 ~ d12, data = df_global)
print(summary(lm_global_shuffle))
```

```
##
## Call:
## lm(formula = d23 ~ d12, data = df_global)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.33328 -0.13516 -0.00757  0.11148  0.36662
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.09094    0.08571   1.061    0.330
## d12         -0.53784    0.45367  -1.186    0.281
##
## Residual standard error: 0.2386 on 6 degrees of freedom
## Multiple R-squared:  0.1898, Adjusted R-squared:  0.05475
## F-statistic: 1.405 on 1 and 6 DF,  p-value: 0.2806
```

3.4 3.3 Turn-Level Permutation Test

```r
# Observed slope
obs_turn_slope <- coef(lm_turns_field)[2]

set.seed(2022)
n_sims <- 1000
perm_slopes_turn <- replicate(n_sims, {
  df_perm <- rb_turns_field %>%
    group_by(conversation_id) %>%
    mutate(gap_change_perm = sample(gap_change)) %>%
    ungroup()
  coef(lm(gap_change_next ~ gap_change_perm, data = df_perm))[2]
})

# Calculate p-value (two-tailed)
p_turn <- mean(abs(perm_slopes_turn) >= abs(obs_turn_slope))

cat("=== Turn-Level Permutation Test Results ===\n")
```

```
## === Turn-Level Permutation Test Results ===
```

```r
cat("Observed slope:", round(obs_turn_slope, 4), "\n")
```

```
## Observed slope: -0.5291
```

```r
cat("Null mean:", round(mean(perm_slopes_turn), 4), "\n")
```

```
## Null mean: -0.0018
```

```r
cat("Null SD:", round(sd(perm_slopes_turn), 4), "\n")
```

```
## Null SD: 0.0551
```

```r
cat("Effect size (z):", round((obs_turn_slope - mean(perm_slopes_turn)) / sd(perm_slopes_turn), 2), "\n
```

```
## Effect size (z): -9.57
```

```r
cat("P-value:", format.pval(p_turn, digits = 3), "\n\n")
```

```
## P-value: <2e-16
```

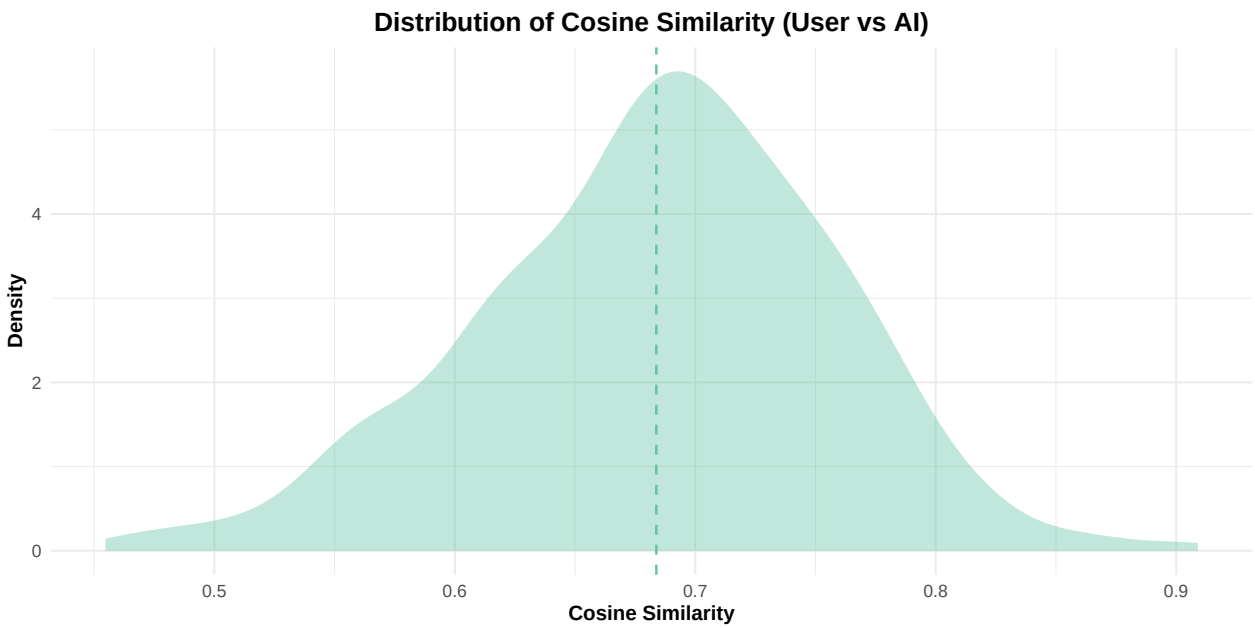
```r
# Tag each turn pair by whether they cross stage boundaries
rb_turns_field %>%
  group_by(conversation_id) %>%
  mutate(
    stage = ntile(turn, 3),
    stage_next = lead(stage)
  ) %>%
  ungroup() %>%
  mutate(
    transition_type = case_when(
      stage == stage_next ~ "Within-Stage",
      stage < stage_next ~ "Stage Boundary",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(transition_type)) %>%
  group_by(transition_type) %>%
  summarise(
    rb_slope = coef(lm(gap_change_next ~ gap_change))[2],
    correlation = cor(gap_change, gap_change_next),
    n = n()
  )
```

```
## # A tibble: 2 x 4
##   transition_type rb_slope correlation     n
##   <chr>              <dbl>       <dbl> <int>
## 1 Stage Boundary    -0.479      -0.472   144
## 2 Within-Stage      -0.591      -0.567   360
```

4 4. Cosine similarity alignment

4.1 4.1 Cosine Similarity Distribution

```r
p_cosine_dist <- ggplot(df_field, aes(x = cosine_sim_same)) +
  geom_density(alpha = 0.4, fill = "#66C2A5", color = NA) +
  geom_vline(aes(xintercept = mean(cosine_sim_same, na.rm = TRUE)),
             color = "#66C2A5", linetype = "dashed", linewidth = 0.8) +
  labs(
    title = "Distribution of Cosine Similarity (User vs AI)",
    x = "Cosine Similarity",
    y = "Density"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

print(p_cosine_dist)
```




```
ggsave(file.path(GRAPHS_DIR, "cosine_similarity_distribution.png"),
       p_cosine_dist, width = 12, height = 6, dpi = 300)
```

4.2 4.2 Cosine Similarity Mixed Model

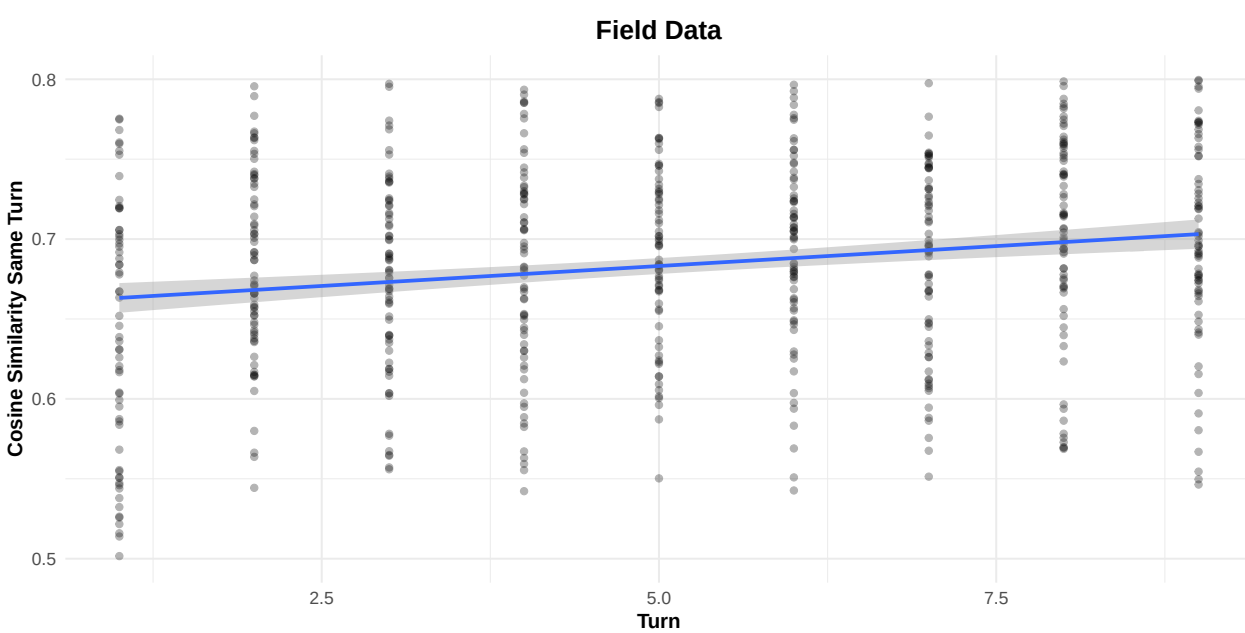
```
df_cosine_sim <- df_field %>%
  filter(turn > 1, turn < 10)

lm_cosine_turn <- lmer(
  cosine_sim_same ~ turn + (1 | conversation_id),
  data = df_cosine_sim
)
print(summary(lm_cosine_turn))
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: cosine_sim_same ~ turn + (1 | conversation_id)
##    Data: df_cosine_sim
##
## REML criterion at convergence: -1520.5
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.64276 -0.66328  0.01881  0.66155  2.87432
##
## Random effects:
##  Groups          Name        Variance  Std.Dev.
##  conversation_id (Intercept) 0.0008528 0.02920
##  Residual                    0.0035357 0.05946
## Number of obs: 576, groups:  conversation_id, 72
##
## Fixed effects:
##              Estimate Std. Error        df t value Pr(>|t|)
## (Intercept) 6.754e-01  7.304e-03 4.042e+02  92.466  < 2e-16 ***
```

```
## turn        3.337e-03  1.081e-03 5.030e+02   3.087  0.00214 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr)
## turn -0.814
```

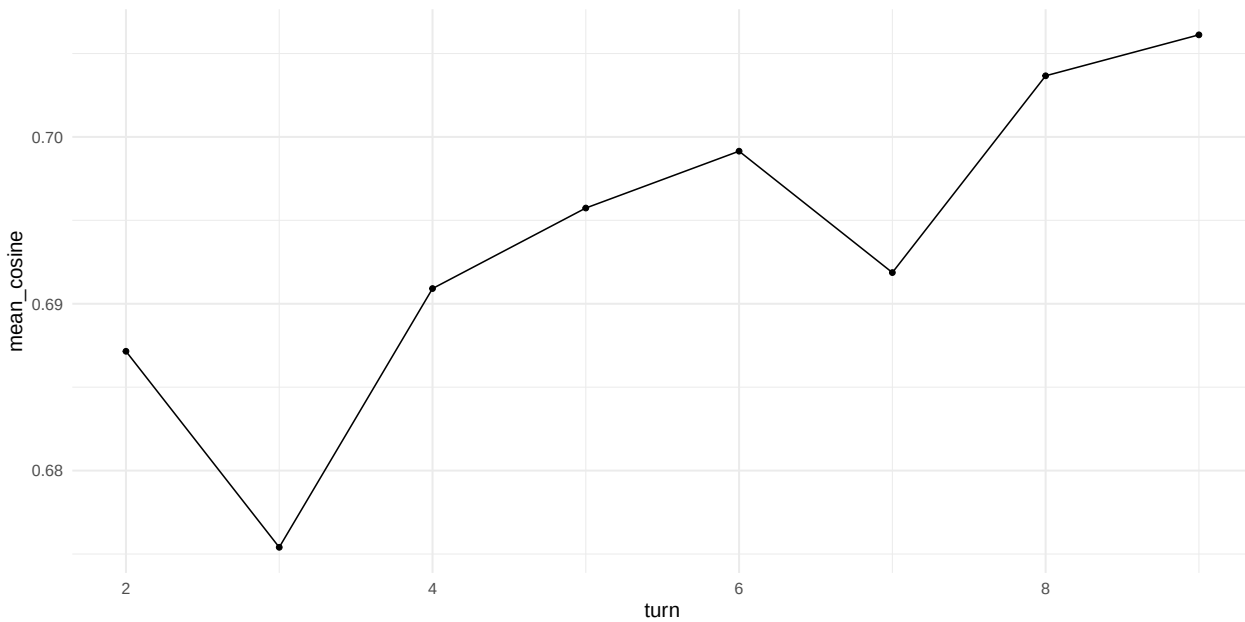
4.3 4.3 Cosine Similarity Linear Plot

```r
cosine_sim_lm_plot <- df_field %>%
  filter(turn > 0, turn < 10) %>%
  ggplot(aes(x = turn, y = cosine_sim_same)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.3, size = 2) +  # no color here
  geom_smooth(method = "lm", se = TRUE, linewidth = 1.2) +  # no color here
  xlim(1, 9) +
  ylim(0.5, 0.8) +
  labs(
    title = "Field Data",
    x = "Turn",
    y = "Cosine Similarity Same Turn"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )
print(cosine_sim_lm_plot)
```

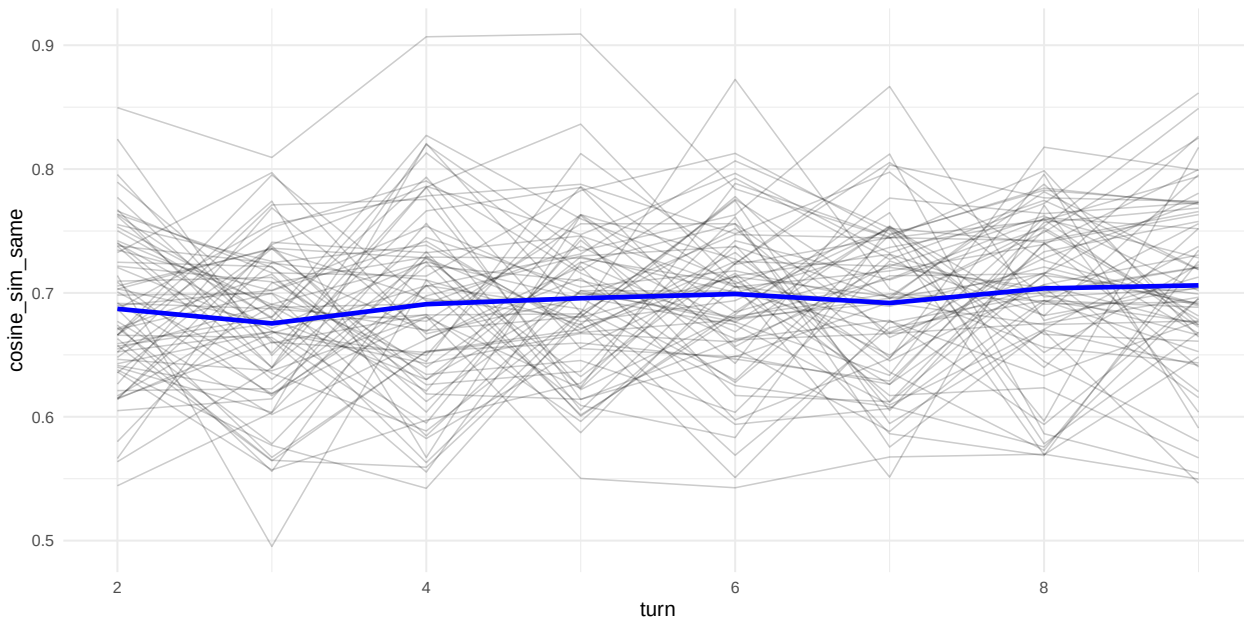


4.4 4.4 Cosine Similarity Over Time

```
df_field %>%
  filter(turn > 1, turn < 10) %>%
  group_by(turn) %>%
  summarise(mean_cosine = mean(cosine_sim_same, na.rm = TRUE)) %>%
  ggplot(aes(x = turn, y = mean_cosine)) +
  geom_line() +
  geom_point()
```




```
df_field %>%
  filter(turn < 10, turn > 1) %>%
  #group_by(starter) %>%
  ggplot(aes(x = turn, y = cosine_sim_same, group = conversation_id)) +
  geom_line(alpha = 0.2) +  # Individual stories (faint)
  stat_summary(aes(group = 1), fun = mean, geom = "line",
               color = "blue", size = 1.5)  # Aggregate mean (bold)
```



4.5 4.5 Cosine rubber band

```
compute_rubberband_cosine_turns <- function(df, data_source_label) {
  df %>%
    group_by(conversation_id) %>%
    arrange(turn, .by_group = TRUE) %>%
    mutate(
      cosine_dist = 1 - cosine_sim_same,
      dist_change = cosine_dist - lag(cosine_dist),  # C fhange from previous turn
      dist_change_next = lead(cosine_dist) - cosine_dist  # Change to next turn
    ) %>%
    filter(!is.na(dist_change) & !is.na(dist_change_next)) %>%  # Need both values
    ungroup() %>%
    mutate(
      data_source = data_source_label,
      n_turns = n_distinct(conversation_id)
    )
}

rb_cosine <- compute_rubberband_cosine_turns(df_field, "Field")
lm_turns_field_cosine <- lm(dist_change_next ~ dist_change, data = rb_cosine)


cat("=== Turn-Level Rubber-Band Cosine Distance: Field Data ===\n")
```

```
## === Turn-Level Rubber-Band Cosine Distance: Field Data ===
```

```
print(summary(lm_turns_field_cosine))
```

```
##
## Call:
## lm(formula = dist_change_next ~ dist_change, data = rb_cosine)
##
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -0.33539 -0.05546 -0.00182  0.05385  0.24815
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.0009926  0.0033734  -0.294    0.769
## dist_change -0.4569420  0.0378966 -12.058   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.08048 on 574 degrees of freedom
## Multiple R-squared:  0.2021, Adjusted R-squared:  0.2007
## F-statistic: 145.4 on 1 and 574 DF,  p-value: < 2.2e-16
```

4.6 4.6 Turn level Cosine Permutation test

```r
# Observed slope
obs_cosine_slope <- coef(lm_turns_field_cosine)[2]

set.seed(2022)
n_sims <- 1000
perm_slopes_cosine <- replicate(n_sims, {
  df_perm <- rb_cosine %>%
    group_by(conversation_id) %>%
    mutate(dist_change_perm = sample(dist_change)) %>%
    ungroup()
  coef(lm(dist_change_next ~ dist_change_perm, data = df_perm))[2]
})

# Calculate p-value (two-tailed)
p_cosine <- mean(abs(perm_slopes_cosine) >= abs(obs_cosine_slope))

cat("=== Cosine Distance Turn-Level Permutation Test Results ===\n")
```

```
## === Cosine Distance Turn-Level Permutation Test Results ===
```

```r
cat("Observed slope:", round(obs_cosine_slope, 4), "\n")
```

```
## Observed slope: -0.4569
```

```r
cat("Null mean:", round(mean(perm_slopes_cosine), 4), "\n")
```

```
## Null mean: -0.0029
```

```r
cat("Null SD:", round(sd(perm_slopes_cosine), 4), "\n")
```

```
## Null SD: 0.0501
```

```r
cat("Effect size (z):", round((obs_cosine_slope - mean(perm_slopes_cosine)) / sd(perm_slopes_cosine), 2
```

```
## Effect size (z): -9.07
```

```r
cat("P-value:", format.pval(p_cosine, digits = 3), "\n\n")
```

```
## P-value: <2e-16
```

```r
# Tag each turn pair by whether they cross stage boundaries
rb_cosine %>%
```

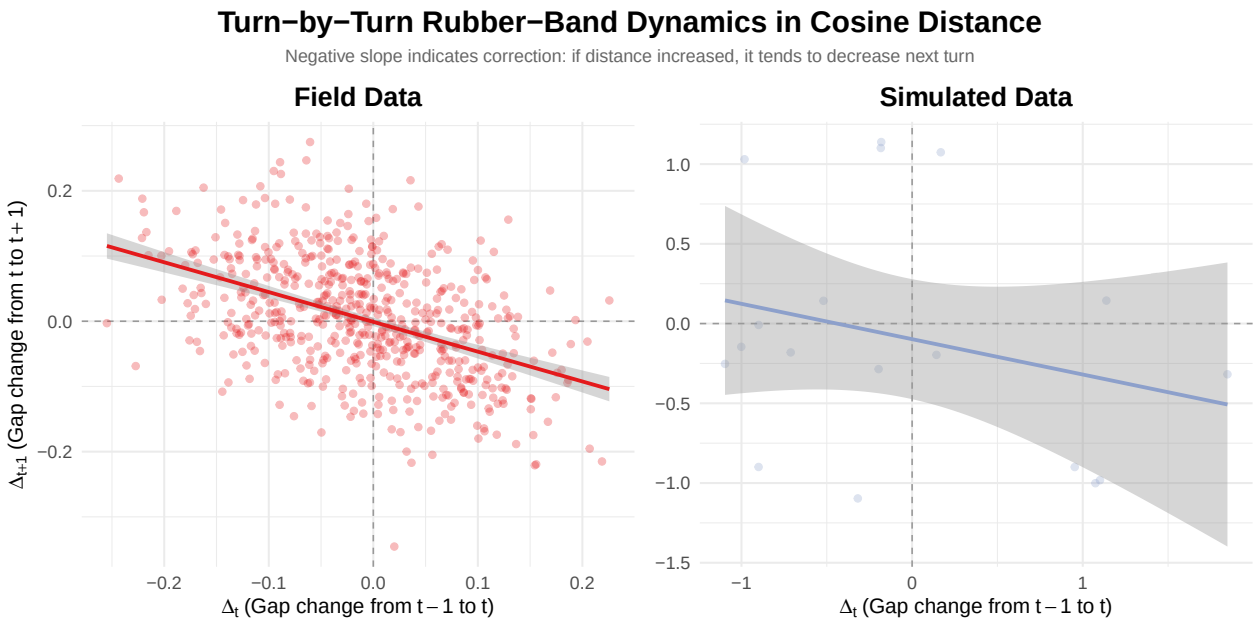
```r
  group_by(conversation_id) %>%
  mutate(
    stage = ntile(turn, 3),
    stage_next = lead(stage)
  ) %>%
  ungroup() %>%
  mutate(
    transition_type = case_when(
      stage == stage_next ~ "Within-Stage",
      stage < stage_next ~ "Stage Boundary",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(transition_type)) %>%
  group_by(transition_type) %>%
  summarise(
    rb_slope = coef(lm(dist_change_next ~ dist_change))[2],
    correlation = cor(dist_change, dist_change_next),
    n = n()
  )
```

```
## # A tibble: 2 x 4
##   transition_type rb_slope correlation     n
##   <chr>              <dbl>       <dbl> <int>
## 1 Stage Boundary    -0.632      -0.628   144
## 2 Within-Stage      -0.384      -0.419   360
```

4.7 4.7 Turn Level Cosine RB Linear plot

```r
p_rb_turns_field_cosine <- ggplot(rb_cosine, aes(x = dist_change, y = dist_change_next)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.3, size = 2, color = "#E41A1C") +
  geom_smooth(method = "lm", se = TRUE, color = "#E41A1C", linewidth = 1.2) +
  labs(
    title = "Field Data",
    x = expression(Delta[t]~"(Gap change from"~t-1~"to"~t*")"),
    y = expression(Delta[t+1]~"(Gap change from"~t~"to"~t+1*")")
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )


combined_rb_turns <- (p_rb_turns_field_cosine | p_rb_turns_sim) +
  plot_annotation(
    title = "Turn-by-Turn Rubber-Band Dynamics in Cosine Distance",
    subtitle = "Negative slope indicates correction: if distance increased, it tends to decrease next tu
    theme = theme(
      plot.title = element_text(hjust = 0.5, size = 20, face = "bold"),
      plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40")
```

```
    )
  )

print(combined_rb_turns)
```




```
ggsave(file.path(GRAPHS_DIR, "rubberband_cosine_turn_level_comparison.png"),
       combined_rb_turns, width = 12, height = 6, dpi = 300)
```

4.8 4.8 Correlation Between Cosine Distance and Valence Rubber-Band

```
# Turn filtering parameters
turn_min <- 2    # Minimum turn to include (set to 1 for no lower bound)
turn_max <- 9  # Maximum turn to include (set to Inf for no upper bound)

# Merge rubber-band data for both metrics
rb_combined <- rb_turns_field %>%
  inner_join(
    rb_cosine %>% select(conversation_id, turn, dist_change, dist_change_next),
    by = c("conversation_id", "turn"),
    suffix = c("_valence", "_cosine")
  ) %>%
  filter(turn >= turn_min, turn <= turn_max)  # Apply turn filtering

# Compute correlation between the two rubber-band patterns
cat("=== Correlation Between Valence and Cosine Distance Rubber-Band ===\n")
```

```
## === Correlation Between Valence and Cosine Distance Rubber-Band ===
```

```
cat("Turn range: [", turn_min, ", ",
    ifelse(is.infinite(turn_max), "Inf", turn_max), "]\n", sep = "")
```

```
## Turn range: [2, 9]
```

```
cat("N observations (turn-level):", nrow(rb_combined), "\n\n")
```

```
## N observations (turn-level): 576
```

```r
# 1. Turn-level correlation of changes (Δt)
# This correlates valence and cosine changes at EACH turn
cor_changes <- cor(rb_combined$gap_change, rb_combined$dist_change,
                   use = "complete.obs")
cat("Turn-level correlation of gap changes (Δt):", round(cor_changes, 3), "\n")
```

```
## Turn-level correlation of gap changes (Δt): 0
```

```r
cat("  Interpretation: Do valence and cosine gaps change in the same direction\n")
```

```
##   Interpretation: Do valence and cosine gaps change in the same direction
```

```r
cat("                  at the same turn across all stories?\n\n")
```

```
##                   at the same turn across all stories?
```

```r
# 2. Turn-level correlation of next changes (Δt+1)
cor_next_changes <- cor(rb_combined$gap_change_next, rb_combined$dist_change_next,
                        use = "complete.obs")
cat("Turn-level correlation of next changes (Δt+1):", round(cor_next_changes, 3), "\n")
```

```
## Turn-level correlation of next changes (Δt+1): 0.012
```

```r
cat("  Interpretation: Do subsequent valence and cosine corrections correlate?\n\n")
```

```
##   Interpretation: Do subsequent valence and cosine corrections correlate?
```

```r
# 3. Per-story rubber-band slopes
# This is the KEY analysis: Do stories with strong valence RB also show strong cosine RB?
rb_slopes <- rb_combined %>%
  group_by(conversation_id) %>%
  filter(n() >= 3) %>%  # Need at least 3 turns for stable regression
  summarise(
    # Valence rubber-band slope:  from (gap_change_next ~ gap_change)
    valence_rb_slope = coef(lm(gap_change_next ~ gap_change))[2],
    # Cosine rubber-band slope:  from (dist_change_next ~ dist_change)
    cosine_rb_slope = coef(lm(dist_change_next ~ dist_change))[2],
    n_turns = n(),
    .groups = "drop"
  )

cat("N stories with >= 3 turns:", nrow(rb_slopes), "\n\n")
```

```
## N stories with >= 3 turns: 72
```

```r
cat("N stories with >= 3 turns:", nrow(rb_slopes), "\n\n")
```

```
## N stories with >= 3 turns: 72
```

```r
# Correlation between story-level slopes
# This answers: Do stories with strong valence RB also show strong cosine RB?
cor_slopes <- cor(rb_slopes$valence_rb_slope, rb_slopes$cosine_rb_slope,
                  use = "complete.obs")
cat("STORY-LEVEL correlation of RB slopes:", round(cor_slopes, 3), "\n")
```

```
## STORY-LEVEL correlation of RB slopes: 0.16
```

```r
cat("  Interpretation: Across stories, does valence RB strength predict cosine RB strength?\n")
```

```
##   Interpretation: Across stories, does valence RB strength predict cosine RB strength?
```

```r
cat("                  Positive r means stories with strong valence correction\n")
```

```
##                    Positive r means stories with strong valence correction
```

```r
cat("                  also show strong cosine distance correction.\n")
```

```
##                    also show strong cosine distance correction.
```

```r
# Test significance
slope_test <- cor.test(rb_slopes$valence_rb_slope, rb_slopes$cosine_rb_slope)
cat("  95% CI: [", round(slope_test$conf.int[1], 3), ", ",
    round(slope_test$conf.int[2], 3), "]\n", sep = "")
```

```
##   95% CI: [-0.075, 0.378]
```

```r
cat("  p-value:", format.pval(slope_test$p.value, digits = 3), "\n")
```

```
##   p-value: 0.18
```

```r
cat("  df:", slope_test$parameter, "\n\n")
```

```
##   df: 70
```

```r
# 4. Shared variance (R²)
shared_var <- cor_slopes^2
cat("Shared variance (R²):", round(shared_var, 3), "\n")
```

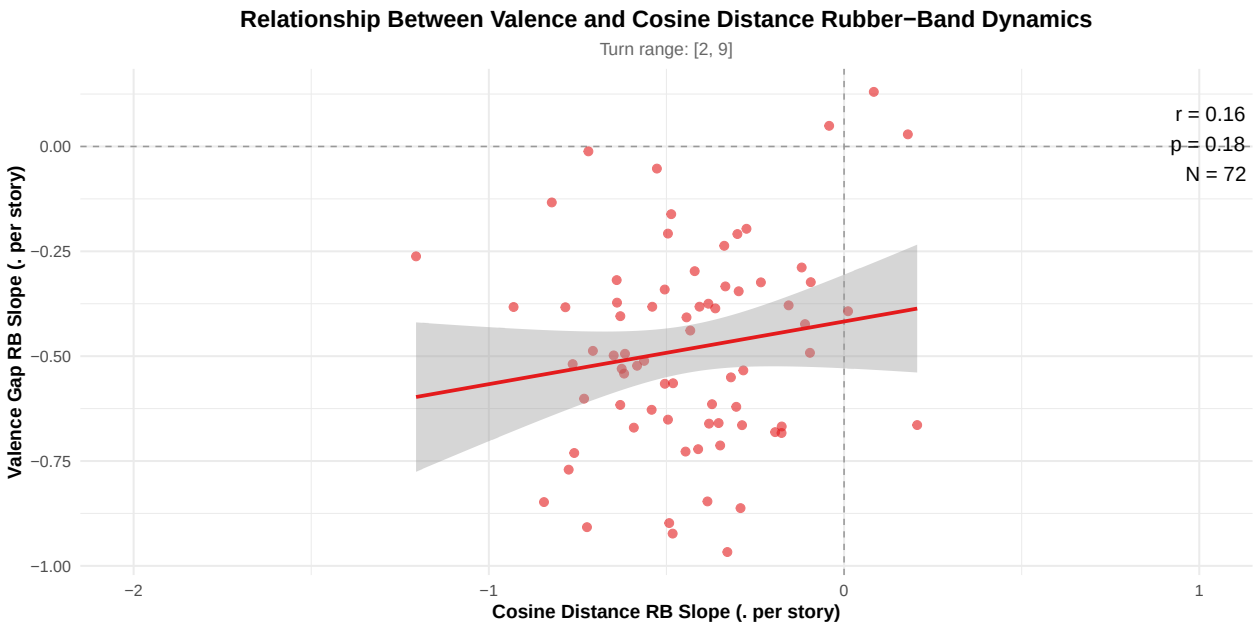
```
## Shared variance (R²): 0.026
```

```r
cat("Interpretation:", round(shared_var * 100, 1),
    "% of variation in valence RB slope\n",
    "               is explained by cosine distance RB slope\n\n")
```

```
## Interpretation: 2.6 % of variation in valence RB slope
##                is explained by cosine distance RB slope
```

```r
# 5. Scatter plot: Story-level RB slopes
p_rb_correlation <- ggplot(rb_slopes, aes(x = cosine_rb_slope, y = valence_rb_slope)) +
  geom_hline(yintercept = 0, linetype = 2, color = "grey60") +
  geom_vline(xintercept = 0, linetype = 2, color = "grey60") +
  geom_point(alpha = 0.6, size = 2.5, color = "#E41A1C") +
  geom_smooth(method = "lm", se = TRUE, color = "#E41A1C", linewidth = 1.2) +
  annotate("text", x = Inf, y = Inf,
           label = paste0("r = ", round(cor_slopes, 3),
                          "\np ", ifelse(slope_test$p.value < 0.001, "< 0.001",
                                         paste("=", round(slope_test$p.value, 3))),
                          "\nN = ", nrow(rb_slopes)),
           hjust = 1.1, vjust = 1.5, size = 5) +
  labs(
    title = "Relationship Between Valence and Cosine Distance Rubber-Band Dynamics",
    subtitle = paste0("Turn range: [", turn_min, ", ",
                      ifelse(is.infinite(turn_max), "∞", turn_max), "]"),
    x = "Cosine Distance RB Slope (  per story)",
    y = "Valence Gap RB Slope (  per story)"
  ) +
```

```r
  xlim(-2,1) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40"),
    axis.title = element_text(size = 13, face = "bold"),
    axis.text = element_text(size = 11)
  )

print(p_rb_correlation)
```




```r
ggsave(file.path(GRAPHS_DIR, "rubberband_valence_cosine_correlation.png"),
       p_rb_correlation, width = 10, height = 8, dpi = 300)

# 6. Regression: Does cosine RB predict valence RB?
lm_rb_predict <- lm(valence_rb_slope ~ cosine_rb_slope, data = rb_slopes)
cat("\n=== Regression: Cosine RB Predicting Valence RB ===\n")
```

```
##
## === Regression: Cosine RB Predicting Valence RB ===
```

```r
print(summary(lm_rb_predict))
```

```
##
## Call:
## lm(formula = valence_rb_slope ~ cosine_rb_slope, data = rb_slopes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.50062 -0.18746  0.01134  0.14198  0.53497
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)     -0.41729    0.05597  -7.455 1.83e-10 ***
## cosine_rb_slope  0.14944    0.11028   1.355     0.18
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2403 on 70 degrees of freedom
## Multiple R-squared:  0.02556,    Adjusted R-squared:  0.01164
## F-statistic: 1.836 on 1 and 70 DF,  p-value: 0.1797
```

```r
cat("\n=== VALIDATION CHECK ===\n")
```

```
##
## === VALIDATION CHECK ===
```

```r
cat("Correlation from cor():", round(cor_slopes, 3), "\n")
```

```
## Correlation from cor(): 0.16
```

```r
cat("Correlation from lm R²:", round(sqrt(summary(lm_rb_predict)$r.squared), 3), "\n")
```

```
## Correlation from lm R²: 0.16
```

```r
cat("These should match (they do if correlation is positive).\n\n")
```

```
## These should match (they do if correlation is positive).
```

5 5. Supplementary: Interaction Effects

5.1 5.1 Starter Interaction Effects

5.1.1 5.1.1 Valence Gap by Starter

```r
# Turn-level valence gap with starter interaction
lm_starter_valence <- lm(gap_change_next ~ gap_change * starter, data = rb_turns_field)

cat("=== Valence Rubber-Band: Starter Interaction ===\n")
```

```
## === Valence Rubber-Band: Starter Interaction ===
```

```r
print(summary(lm_starter_valence))
```

```
##
## Call:
## lm(formula = gap_change_next ~ gap_change * starter, data = rb_turns_field)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.12323 -0.28255 -0.03689  0.27005  1.57933
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)             0.007928   0.022759   0.348    0.728
## gap_change             -0.512635   0.049568 -10.342   <2e-16 ***
## starteruser            -0.008030   0.035014  -0.229    0.819
## gap_change:starteruser -0.044307   0.072325  -0.613    0.540
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4121 on 564 degrees of freedom
##   (8 observations deleted due to missingness)
## Multiple R-squared:  0.2796, Adjusted R-squared:  0.2757
```

```
## F-statistic: 72.95 on 3 and 564 DF,  p-value: < 2.2e-16
```

5.1.2 5.1.2 Cosine Distance by Starter

```r
# Turn-level cosine distance with starter interaction
lm_starter_cosine <- lm(dist_change_next ~ dist_change * starter, data = rb_cosine)

cat("=== Cosine Distance Rubber-Band: Starter Interaction ===\n")
```

```
## === Cosine Distance Rubber-Band: Starter Interaction ===
```

```r
print(summary(lm_starter_cosine))
```

```
##
## Call:
## lm(formula = dist_change_next ~ dist_change * starter, data = rb_cosine)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.33402 -0.05579 -0.00182  0.05535  0.24689
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             -0.001705   0.004512  -0.378    0.706
## dist_change             -0.489557   0.048297 -10.136   <2e-16 ***
## starteruser              0.001085   0.006895   0.157    0.875
## dist_change:starteruser  0.075002   0.079408   0.945    0.345
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.08072 on 564 degrees of freedom
##   (8 observations deleted due to missingness)
## Multiple R-squared:  0.2062, Adjusted R-squared:  0.202
## F-statistic: 48.83 on 3 and 564 DF,  p-value: < 2.2e-16
```

5.1.3 5.1.3 Alignment (Cosine Similarity) by Starter

```r
# Cosine similarity over turns with starter interaction
df_alignment_starter <- df_field %>%
  filter(turn >= 2, turn <= 9, !is.na(cosine_sim_same), !is.na(starter))

lm_alignment_starter <- lm(cosine_sim_same ~ turn * starter, data = df_alignment_starter)

cat("=== Cosine Similarity (Alignment): Starter × Turn Interaction ===\n")
```

```
## === Cosine Similarity (Alignment): Starter × Turn Interaction ===
```

```r
print(summary(lm_alignment_starter))
```

```
##
## Call:
## lm(formula = cosine_sim_same ~ turn * starter, data = df_alignment_starter)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.186678 -0.039645  0.002228  0.044010  0.181815
```

```
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)       0.673311   0.009257  72.732   <2e-16 ***
## turn              0.002880   0.001554   1.853   0.0643 .
## starteruser      -0.002611   0.014242  -0.183   0.8546
## turn:starteruser  0.001693   0.002390   0.708   0.4792
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06448 on 564 degrees of freedom
## Multiple R-squared:  0.01959,    Adjusted R-squared:  0.01437
## F-statistic: 3.756 on 3 and 564 DF,  p-value: 0.01086
```

5.2 5.2 LLM Type Interaction Effects

5.2.1 5.2.1 Valence Gap by LLM Type

```r
# Turn-level valence gap with LLM type interaction
rb_turns_llm <- rb_turns_field %>%
  filter(!is.na(llm_type))

lm_llm_valence <- lm(gap_change_next ~ gap_change * factor(llm_type), data = rb_turns_llm)

cat("=== Valence Rubber-Band: LLM Type Interaction ===\n")
```

```
## === Valence Rubber-Band: LLM Type Interaction ===
```

```r
print(summary(lm_llm_valence))
```

```
##
## Call:
## lm(formula = gap_change_next ~ gap_change * factor(llm_type),
##     data = rb_turns_llm)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.1026 -0.2771 -0.0299  0.2639  1.5681
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   0.0009788  0.0379483   0.026    0.979
## gap_change                   -0.5151532  0.0788254  -6.535 1.41e-10 ***
## factor(llm_type)2            -0.0041404  0.0528215  -0.078    0.938
## factor(llm_type)3             0.0104315  0.0496880   0.210    0.834
## factor(llm_type)4             0.0119904  0.0502228   0.239    0.811
## gap_change:factor(llm_type)2 -0.0381239  0.1131211  -0.337    0.736
## gap_change:factor(llm_type)3  0.0555432  0.1078676   0.515    0.607
## gap_change:factor(llm_type)4 -0.0565111  0.0994750  -0.568    0.570
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4157 on 568 degrees of freedom
## Multiple R-squared:  0.2764, Adjusted R-squared:  0.2675
## F-statistic:    31 on 7 and 568 DF,  p-value: < 2.2e-16
```

5.2.2 5.2.2 Cosine Distance by LLM Type

```r
# Turn-level cosine distance with LLM type interaction
rb_cosine_llm <- rb_cosine %>%
  filter(!is.na(llm_type))

lm_llm_cosine <- lmer(dist_change_next ~ dist_change * factor(llm_type) + (1 | conversation_id), data =

cat("=== Cosine Distance Rubber-Band: LLM Type Interaction ===\n")
```

```
## === Cosine Distance Rubber-Band: LLM Type Interaction ===
```

```r
print(summary(lm_llm_cosine))
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula:
## dist_change_next ~ dist_change * factor(llm_type) + (1 | conversation_id)
##    Data: rb_cosine_llm
##
## REML criterion at convergence: -1227
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.1490 -0.6656 -0.0344  0.6407  3.0137
##
## Random effects:
##  Groups          Name        Variance Std.Dev.
##  conversation_id (Intercept) 0.000000 0.00000
##  Residual                    0.006514 0.08071
## Number of obs: 576, groups:  conversation_id, 72
##
## Fixed effects:
##                                 Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)                    6.561e-04  7.413e-03  5.680e+02   0.089    0.930
## dist_change                   -5.108e-01  7.527e-02  5.680e+02  -6.786  2.9e-11
## factor(llm_type)2              4.081e-03  1.029e-02  5.680e+02   0.397    0.692
## factor(llm_type)3             -5.481e-03  9.739e-03  5.680e+02  -0.563    0.574
## factor(llm_type)4             -3.565e-03  9.812e-03  5.680e+02  -0.363    0.717
## dist_change:factor(llm_type)2  6.412e-02  1.135e-01  5.680e+02   0.565    0.572
## dist_change:factor(llm_type)3  2.904e-02  1.025e-01  5.680e+02   0.283    0.777
## dist_change:factor(llm_type)4  1.235e-01  1.077e-01  5.680e+02   1.147    0.252
##
## (Intercept)
## dist_change                   ***
## factor(llm_type)2
## factor(llm_type)3
## factor(llm_type)4
## dist_change:factor(llm_type)2
## dist_change:factor(llm_type)3
## dist_change:factor(llm_type)4
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
```

```
##             (Intr) dst_ch fc(_)2 fc(_)3 fc(_)4 d_:(_)2 d_:(_)3
## dist_change  0.110
## fctr(llm_)2 -0.721 -0.079
## fctr(llm_)3 -0.761 -0.084  0.548
## fctr(llm_)4 -0.756 -0.083  0.544  0.575
## dst_ch:(_)2 -0.073 -0.663  0.058  0.056  0.055
## dst_ch:(_)3 -0.081 -0.734  0.058  0.135  0.061  0.487
## dst_ch:(_)4 -0.077 -0.699  0.056  0.059  0.115  0.464   0.513
## optimizer (nloptwrap) convergence code: 0 (OK)
## boundary (singular) fit: see help('isSingular')
```

5.2.3 5.2.3 Alignment (Cosine Similarity) by LLM Type

```r
# Cosine similarity over turns with LLM type interaction
df_alignment_llm <- df_field %>%
  filter(turn >= 1, turn <= 5, !is.na(cosine_sim_same), !is.na(llm_type))

lm_alignment_llm <- lmer(cosine_sim_same ~ turn * factor(llm_type) + (1 | conversation_id), data = df_al

cat("=== Cosine Similarity (Alignment): LLM Type × Turn Interaction ===\n")
```

```
## === Cosine Similarity (Alignment): LLM Type × Turn Interaction ===
```

```r
print(summary(lm_alignment_llm))
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: cosine_sim_same ~ turn * factor(llm_type) + (1 | conversation_id)
##    Data: df_alignment_llm
##
## REML criterion at convergence: -807.6
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.70577 -0.57208  0.03232  0.62263  3.08889
##
## Random effects:
##  Groups          Name        Variance  Std.Dev.
##  conversation_id (Intercept) 0.0005795 0.02407
##  Residual                    0.0048305 0.06950
## Number of obs: 360, groups:  conversation_id, 72
##
## Fixed effects:
##                          Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)              0.620328   0.019821 341.540798  31.297   <2e-16 ***
## turn                     0.012355   0.005675 284.000001   2.177   0.0303 *
## factor(llm_type)2        0.067987   0.027589 341.540798   2.464   0.0142 *
## factor(llm_type)3       -0.007590   0.025952 341.540798  -0.292   0.7701
## factor(llm_type)4        0.002827   0.026221 341.540798   0.108   0.9142
## turn:factor(llm_type)2  -0.009284   0.007899 284.000000  -1.175   0.2408
## turn:factor(llm_type)3   0.009183   0.007430 284.000000   1.236   0.2175
## turn:factor(llm_type)4   0.003128   0.007507 284.000000   0.417   0.6772
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Correlation of Fixed Effects:
##             (Intr) turn   fc(_)2 fc(_)3 fc(_)4 t:(_)2 t:(_)3
## turn        -0.859
## fctr(llm_)2 -0.718  0.617
## fctr(llm_)3 -0.764  0.656  0.549
## fctr(llm_)4 -0.756  0.649  0.543  0.577
## trn:fct(_)2  0.617 -0.718 -0.859 -0.471 -0.466
## trn:fct(_)3  0.656 -0.764 -0.471 -0.859 -0.496  0.549
## trn:fct(_)4  0.649 -0.756 -0.466 -0.496 -0.859  0.543  0.577
```